```
/*      A sometimes minimal FORTH compiler and tutorial for Linux / i386 systems. -*- asm -*-
        By Richard W.M. Jones <rich@annexia.org> http://annexia.org/forth
        This is PUBLIC DOMAIN (see public domain release statement below).
        $Id: jonesforth.S,v 1.47 2009-09-11 08:33:13 rich Exp $

        gcc -m32 -nostdlib -static -Wl,-Ttext,0 -Wl,--build-id=none -o jonesforth jonesforth.S
*/
        .set JONES_VERSION,47
/*
        INTRODUCTION ----------------------------------------------------------------------

        FORTH is one of those alien languages which most working programmers regard in the same
        way as Haskell, LISP, and so on.  Something so strange that they'd rather any thoughts
        of it just go away so they can get on with writing this paying code.  But that's wrong
        and if you care at all about programming then you should at least understand all these
        languages, even if you will never use them.

        LISP is the ultimate high-level language, and features from LISP are being added every
        decade to the more common languages.  But FORTH is in some ways the ultimate in low level
        programming.  Out of the box it lacks features like dynamic memory management and even
        strings.  In fact, at its primitive level it lacks even basic concepts like IF-statements
        and loops.

        Why then would you want to learn FORTH?  There are several very good reasons.  First
        and foremost, FORTH is minimal.  You really can write a complete FORTH in, say, 2000
        lines of code.  I don't just mean a FORTH program, I mean a complete FORTH operating
        system, environment and language.  You could boot such a FORTH on a bare PC and it would
        come up with a prompt where you could start doing useful work.  The FORTH you have here
        isn't minimal and uses a Linux process as its 'base PC' (both for the purposes of making
        it a good tutorial). It's possible to completely understand the system.  Who can say they
        completely understand how Linux works, or gcc?

        Secondly FORTH has a peculiar bootstrapping property.  By that I mean that after writing
        a little bit of assembly to talk to the hardware and implement a few primitives, all the
        rest of the language and compiler is written in FORTH itself.  Remember I said before
        that FORTH lacked IF-statements and loops?  Well of course it doesn't really because
        such a lanuage would be useless, but my point was rather that IF-statements and loops are
        written in FORTH itself.

        Now of course this is common in other languages as well, and in those languages we call
        them 'libraries'.  For example in C, 'printf' is a library function written in C.  But
        in FORTH this goes way beyond mere libraries.  Can you imagine writing C's 'if' in C?
        And that brings me to my third reason: If you can write 'if' in FORTH, then why restrict
        yourself to the usual if/while/for/switch constructs?  You want a construct that iterates
        over every other element in a list of numbers?  You can add it to the language.  What
        about an operator which pulls in variables directly from a configuration file and makes
        them available as FORTH variables?  Or how about adding Makefile-like dependencies to
        the language?  No problem in FORTH.  How about modifying the FORTH compiler to allow
        complex inlining strategies -- simple.  This concept isn't common in programming languages,
```

but it has a name (in fact two names): "macros" (by which I mean LISP-style macros, not the lame C preprocessor) and "domain specific languages" (DSLs).

This tutorial isn't about learning FORTH as the language. I'll point you to some references you should read if you're not familiar with using FORTH. This tutorial is about how to write FORTH. In fact, until you understand how FORTH is written, you'll have only a very superficial understanding of how to use it.

So if you're not familiar with FORTH or want to refresh your memory here are some online references to read:

http://en.wikipedia.org/wiki/Forth%28programming language%29

http://galileo.phys.virginia.edu/classes/551.jvn.fall01/primer.htm

http://wiki.laptop.org/go/Forth Lessons

http://www.albany.net/hello/simple.htm

Here is another "Why FORTH?" essay: http://www.jwdt.com/paysan/why-forth.html

Discussion and criticism of this FORTH here: http://lambda-the-ultimate.org/node/2452

ACKNOWLEDGEMENTS ----------

This code draws heavily on the design of LINA FORTH (http://home.hccnet.nl/a.w.m.van.der.horst/lina.html) by Albert van der Horst. Any similarities in the code are probably not accidental.

Some parts of this FORTH are also based on this IOCCC entry from 1992:
http://ftp.funet.fi/pub/doc/IOCCC/1992/buzzard.2.design.
I was very proud when Sean Barrett, the original author of the IOCCC entry, commented in the LtU thread http://lambda-the-ultimate.org/node/2452#comment-36818 about this FORTH.

And finally I'd like to acknowledge the (possibly forgotten?) authors of ARTIC FORTH because their original program which I still have on original cassette tape kept nagging away at me all these years.
http://en.wikipedia.org/wiki/Artic Software

PUBLIC DOMAIN ----------

I, the copyright holder of this work, hereby release it into the public domain. This applies worldwide.

In case this is not legally possible, I grant any entity the right to use this work for any purpose, without any conditions, unless such conditions are required by law.

SETTING UP ----------

Let's get a few housekeeping things out of the way. Firstly because I need to draw lots of ASCII-art diagrams to explain concepts, the best way to look at this is using a window which uses a fixed width font and is at least this wide:

```
99
100
    <-------------------------------------------------------------------------------------------------------------
    >
101
102         Secondly make sure TABS are set to 8 characters.  The following should be a vertical
103         line.  If not, sort out your tabs.
104
105                 |
106                 |
107                 |
108
109         Thirdly I assume that your screen is at least 50 characters high.
110
111         ASSEMBLING ----------------------------------------------------------------------
112
113         If you want to actually run this FORTH, rather than just read it, you will need Linux on an
114         i386.  Linux because instead of programming directly to the hardware on a bare PC which I
115         could have done, I went for a simpler tutorial by assuming that the 'hardware' is a Linux
116         process with a few basic system calls (read, write and exit and that's about all).  i386
117         is needed because I had to write the assembly for a processor, and i386 is by far the most
118         common.  (Of course when I say 'i386', any 32- or 64-bit x86 processor will do.  I'm compiling
119         this on a 64 bit AMD Opteron).
120
121         Again, to assemble this you will need gcc and gas (the GNU assembler).  The commands to
122         assemble and run the code (save this file as 'jonesforth.S') are:
123
124         gcc -m32 -nostdlib -static -Wl,-Ttext,0 -Wl,--build-id=none -o jonesforth jonesforth.S
125         cat jonesforth.f - | ./jonesforth
126
127         If you want to run your own FORTH programs you can do:
128
129         cat jonesforth.f myprog.f | ./jonesforth
130
131         If you want to load your own FORTH code and then continue reading user commands, you can do:
132
133         cat jonesforth.f myfunctions.f - | ./jonesforth
134
135         ASSEMBLER -----------------------------------------------------------------------
136
137         (You can just skip to the next section -- you don't need to be able to read assembler to
138         follow this tutorial).
139
140         However if you do want to read the assembly code here are a few notes about gas (the GNU assembler):
141
142         (1) Register names are prefixed with '%', so %eax is the 32 bit i386 accumulator.  The registers
143             available on i386 are: %eax, %ebx, %ecx, %edx, %esi, %edi, %ebp and %esp, and most of them
144             have special purposes.
145
```

(2) Add, mov, etc. take arguments in the form SRC,DEST. So mov %eax,%ecx moves %eax -> %ecx

(3) Constants are prefixed with '$', and you mustn't forget it! If you forget it then it
    causes a read from memory instead, so:
    mov $2,%eax   moves number 2 into %eax
    mov 2,%eax   reads the 32 bit word from address 2 into %eax (ie. most likely a mistake)

(4) gas has a funky syntax for local labels, where '1f' (etc.) means label '1:' "forwards"
    and '1b' (etc.) means label '1:' "backwards". Notice that these labels might be mistaken
    for hex numbers (eg. you might confuse 1b with $0x1b).

(5) 'ja' is "jump if above", 'jb' for "jump if below", 'je' "jump if equal" etc.

(6) gas has a reasonably nice .macro syntax, and I use them a lot to make the code shorter and
    less repetitive.

For more help reading the assembler, do "info gas" at the Linux prompt.

Now the tutorial starts in earnest.

THE DICTIONARY --

In FORTH as you will know, functions are called "words", and just as in other languages they
have a name and a definition. Here are two FORTH words:

```
: DOUBLE DUP + ;        \ name is "DOUBLE", definition is "DUP +"
: QUADRUPLE DOUBLE DOUBLE ;     \ name is "QUADRUPLE", definition is "DOUBLE DOUBLE"
```

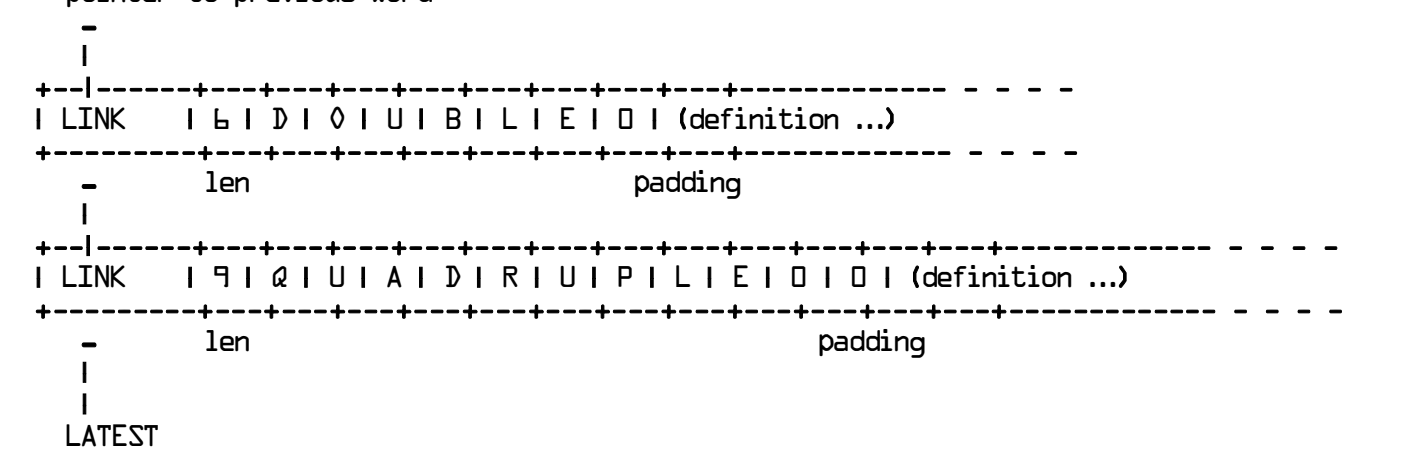
Words, both built-in ones and ones which the programmer defines later, are stored in a dictionary
which is just a linked list of dictionary entries.

```
<--- DICTIONARY ENTRY (HEADER) ---------------------->
+-----------------------+--------+---------- - - - - +----------- - - - -
| LINK POINTER          | LENGTH/| NAME      | DEFINITION
|                       | FLAGS  |           |
+--- (4 bytes) ----------+- byte -+- n bytes  - - - - +----------- - - - -
```

I'll come to the definition of the word later. For now just look at the header. The first
4 bytes are the link pointer. This points back to the previous word in the dictionary, or, for
the first word in the dictionary it is just a NULL pointer. Then comes a length/flags byte.
The length of the word can be up to 31 characters (5 bits used) and the top three bits are used
for various flags which I'll come to later. This is followed by the name itself, and in this
implementation the name is rounded up to a multiple of 4 bytes by padding it with zero bytes.
That's just to ensure that the definition starts on a 32 bit boundary.

A FORTH variable called LATEST contains a pointer to the most recently defined word, in
other words, the head of this linked list.

DOUBLE and QUADRUPLE might look like this:



You should be able to see from this how you might implement functions to find a word **in** the dictionary (just walk along the dictionary entries starting at LATEST and matching the names until you either find a match or hit the **NULL** pointer at the end of the dictionary); and add a word to the dictionary (create a new definition, set its LINK to LATEST, and set LATEST to point to the new word). We'll see precisely these functions implemented in assembly code later on.

One interesting consequence of using a linked list is that you can redefine words, and a newer definition of a word overrides an older one. This is an important concept in FORTH because it means that any word (even "built-in" or "standard" words) can be overridden with a new definition, either to enhance it, to make it faster or even to disable it. However because of the way that FORTH words get compiled, which you'll understand below, words defined using the old definition of a word continue to use the old definition. Only words defined after the new definition use the new definition.

DIRECT THREADED CODE

Now we'll get to the really crucial bit in understanding FORTH, so go and get a cup of tea or coffee and settle down. It's fair to say that **if** you don't understand this section, then you won't "get" how FORTH works, and that would be a failure on my part **for** not explaining it well. So **if** after reading this section a few times you don't understand it, please email me (rich@annexia.org).

Let's talk first about what "threaded code" means. Imagine a peculiar version of C where you are only allowed to call functions without arguments. (Don't worry for now that such a language would be completely useless!) So in our peculiar C, code would look like this:

```
f ()
{
  a ();
  b ();
  c ();
```

```
}
```

and so on. How would a function, say 'f' above, be compiled by a standard C compiler?
Probably into assembly code like this. On the right hand side I've written the actual
i386 machine code.

```
f:
  CALL a                        E8 08 00 00 00
  CALL b                        E8 1C 00 00 00
  CALL c                        E8 2C 00 00 00
  ; ignore the return from the function for now
```

"E8" is the x86 machine code to "CALL" a function. In the first 20 years of computing
memory was hideously expensive and we might have worried about the wasted space being used
by the repeated "E8" bytes. We can save 20% in code size (and therefore, in expensive memory)
by compressing this into just:

```
08 00 00 00             Just the function addresses, without
1C 00 00 00             the CALL prefix.
2C 00 00 00
```

On a 16-bit machine like the ones which originally ran FORTH the savings are even greater - 33%.

[Historical note: If the execution model that FORTH uses looks strange from the following
paragraphs, then it was motivated entirely by the need to save memory on early computers.
This code compression isn't so important now when our machines have more memory in their L1
caches than those early computers had in total, but the execution model still has some
useful properties].

Of course this code won't run directly on the CPU any more. Instead we need to write an
interpreter which takes each set of bytes and calls it.

On an i386 machine it turns out that we can write this interpreter rather easily, in just
two assembly instructions which turn into just 3 bytes of machine code. Let's store the
pointer to the next word to execute in the %esi register:

```
        08 00 00 00     <- We're executing this one now.  %esi is the next one to execute.
%esi -> 1C 00 00 00
        2C 00 00 00
```

The all-important i386 instruction is called LODSL (or in Intel manuals, LODSW). It does
two things. Firstly it reads the memory at %esi into the accumulator (%eax). Secondly it
increments %esi by 4 bytes. So after LODSL, the situation now looks like this:

```
        08 00 00 00     <- We're still executing this one
        1C 00 00 00     <- %eax now contains this address (0x0000001C)
%esi -> 2C 00 00 00
```

Now we just need to jump to the address in %eax. This is again just a single x86 instruction

```
293         written JMP *(%eax).  And after doing the jump, the situation looks like:
294
295                 08 00 00 00
296                 1C 00 00 00    <- Now we're executing this subroutine.
297         %esi -> 2C 00 00 00
298
299         To make this work, each subroutine is followed by the two instructions 'LODSL; JMP *(%eax)'
300         which literally make the jump to the next subroutine.
301
302         And that brings us to our first piece of actual code!  Well, it's a macro.
303 */
304
305 /* NEXT macro. */
306         .macro NEXT
307         lodsl
308         jmp *(%eax)
309         .endm
310
311 /*      The macro is called NEXT.  That's a FORTH-ism.  It expands to those two instructions.
312
313         Every FORTH primitive that we write has to be ended by NEXT.  Think of it kind of like
314         a return.
315
316         The above describes what is known as direct threaded code.
317
318         To sum up: We compress our function calls down to a list of addresses and use a somewhat
319         magical macro to act as a "jump to next function in the list".  We also use one register (%esi)
320         to act as a kind of instruction pointer, pointing to the next function in the list.
321
322         I'll just give you a hint of what is to come by saying that a FORTH definition such as:
323
324         : QUADRUPLE DOUBLE DOUBLE ;
325
326         actually compiles (almost, not precisely but we'll see why in a moment) to a list of
327         function addresses for DOUBLE, DOUBLE and a special function called EXIT to finish off.
328
329         At this point, REALLY EAGLE-EYED ASSEMBLY EXPERTS are saying "JONES, YOU'VE MADE A MISTAKE!".
330
331         I lied about JMP *(%eax).
332
333         INDIRECT THREADED CODE ----------------------------------------------------------------------
334
335         It turns out that direct threaded code is interesting but only if you want to just execute
336         a list of functions written in assembly language.  So QUADRUPLE would work only if DOUBLE
337         was an assembly language function.  In the direct threaded code, QUADRUPLE would look like:
338
339                 +------------------+
340                 | addr of DOUBLE  ---------------------> (assembly code to do the double)
341                 +------------------+                      NEXT
```

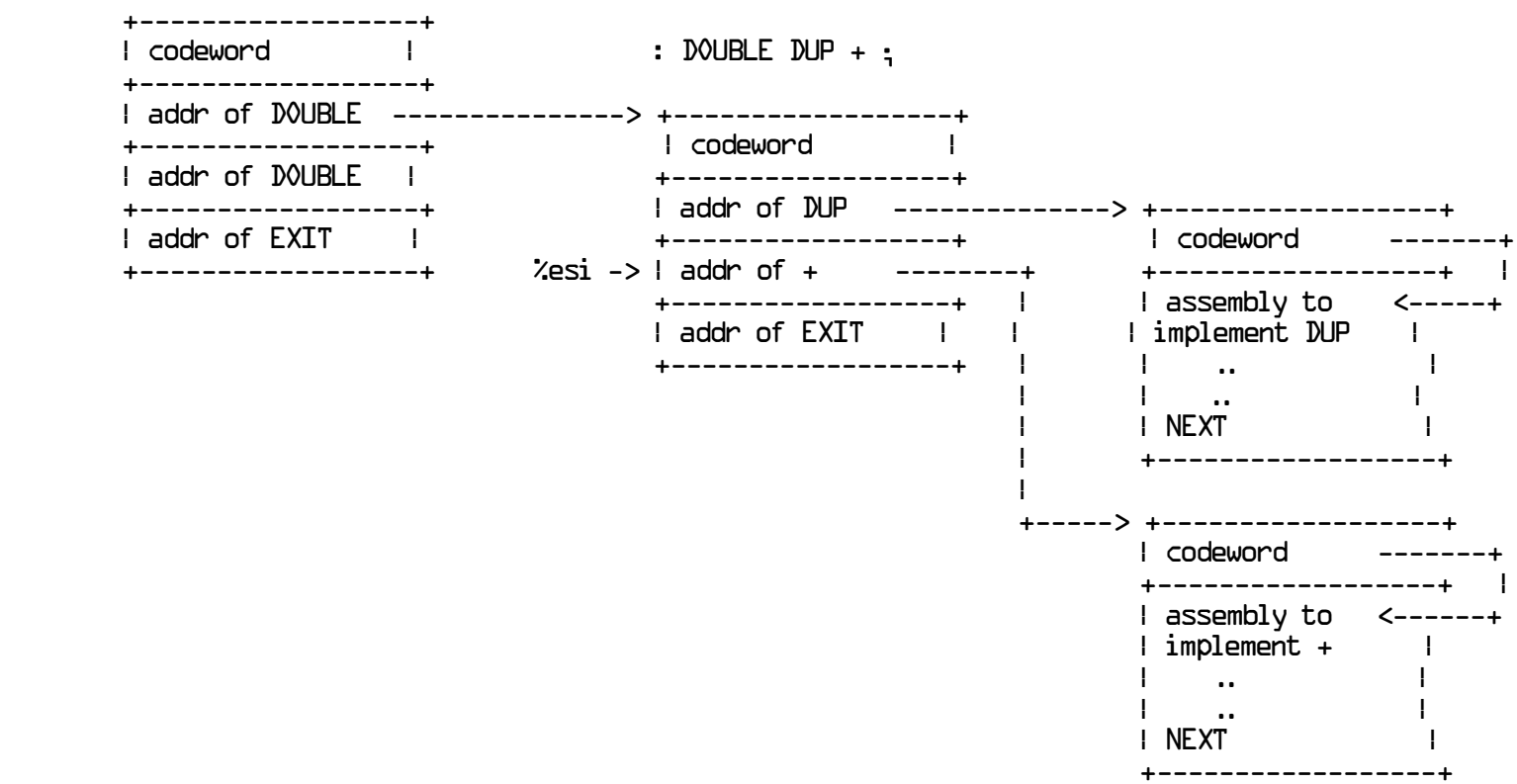
```
%esi -> | addr of DOUBLE   |
        +------------------+
```

We can add an extra indirection to allow us to run both words written **in** assembly language (primitives written **for** speed) and words written **in** FORTH themselves as lists of addresses.

The extra indirection is the reason **for** the brackets **in** JMP *(%eax).

Let's have a look at how QUADRUPLE and DOUBLE really look in FORTH:

```
: QUADRUPLE DOUBLE DOUBLE ;
```



This is the part where you may need an extra cup of tea/coffee/favourite caffeinated beverage. What has changed is that I've added an extra pointer to the beginning of the definitions. In FORTH this is sometimes called the "codeword". The codeword is a pointer to the interpreter to run the function. For primitives written **in** assembly language, the "interpreter" just points to the actual assembly code itself. They don't need interpreting, they just run.

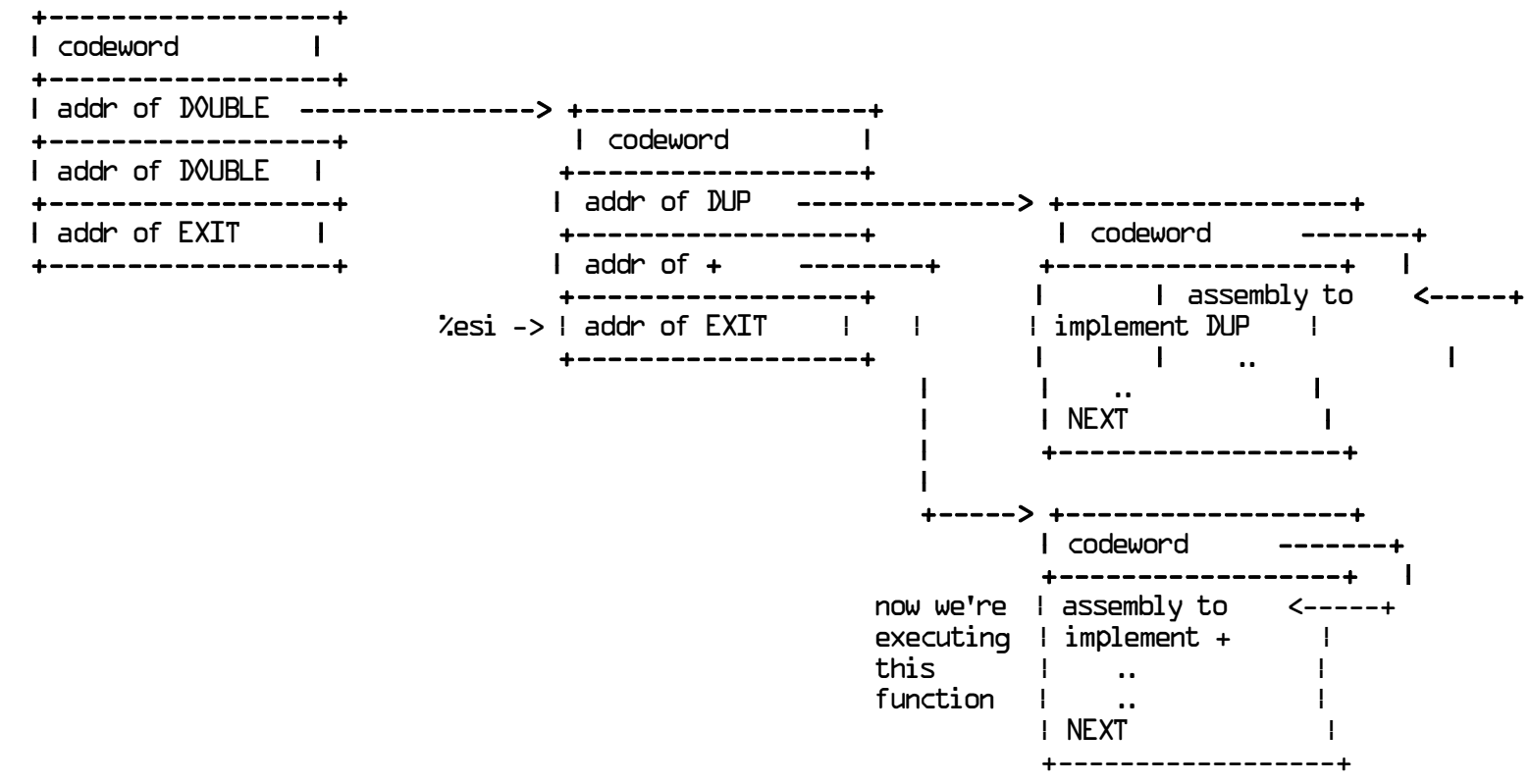
In words written in FORTH (like QUADRUPLE and DOUBLE), the codeword points to an interpreter function.

I'll show you the interpreter **function** shortly, but let's recall our indirect

JMP *(%eax) with the "extra" brackets. Take the case where we're executing DOUBLE as shown, and DUP has been called. Note that %esi is pointing to the address of +

The assembly code **for** DUP eventually does a NEXT. That:

(1) reads the address of + into %eax — %eax points to the codeword of +
(2) increments %esi by 4
(3) jumps to the indirect %eax — jumps to the address in the codeword of +, ie. the assembly code to implement +



So I hope that I've convinced you that NEXT does roughly what you'd expect. This is indirect threaded code.

I've glossed over four things. I wonder **if** you can guess without reading on what they are?

.
.
.

My list of four things are: (1) What does "EXIT" do? (2) which is related to (1) is how do you call into a **function**, ie. how does %esi start off pointing at part of QUADRUPLE, but then point at part of DOUBLE. (3) What goes **in** the codeword **for** the words which are written **in** FORTH? (4) How do you compile a **function** which does anything except call other functions

```
        ie. a function which contains a number like : DOUBLE 2 * ; ?

        THE INTERPRETER AND RETURN STACK ----------------------------------------------------------

        Going at these in no particular order, let's talk about issues (3) and (2), the interpreter
        and the return stack.

        Words which are defined in FORTH need a codeword which points to a little bit of code to
        give them a "helping hand" in life.  They don't need much, but they do need what is known
        as an "interpreter", although it doesn't really "interpret" in the same way that, say,
        Java bytecode used to be interpreted (ie. slowly).  This interpreter just sets up a few
        machine registers so that the word can then execute at full speed using the indirect
        threaded model above.

        One of the things that needs to happen when QUADRUPLE calls DOUBLE is that we save the old
        %esi ("instruction pointer") and create a new one pointing to the first word in DOUBLE.
        Because we will need to restore the old %esi at the end of DOUBLE (this is, after all, like
        a function call), we will need a stack to store these "return addresses" (old values of %esi).

        As you will have seen in the background documentation, FORTH has two stacks, an ordinary
        stack for parameters, and a return stack which is a bit more mysterious.  But our return
        stack is just the stack I talked about in the previous paragraph, used to save %esi when
        calling from a FORTH word into another FORTH word.

        In this FORTH, we are using the normal stack pointer (%esp) for the parameter stack.
        We will use the i386's "other" stack pointer (%ebp, usually called the "frame pointer")
        for our return stack.

        I've got two macros which just wrap up the details of using %ebp for the return stack.
        You use them as for example "PUSHRSP %eax" (push %eax on the return stack) or "POPRSP %ebx"
        (pop top of return stack into %ebx).
*/

/* Macros to deal with the return stack. */
        .macro PUSHRSP reg
        lea -4(%ebp),%ebp       // push reg on to return stack
        movl \reg,(%ebp)
        .endm

        .macro POPRSP reg
        mov (%ebp),\reg         // pop top of return stack to reg
        lea 4(%ebp),%ebp
        .endm

/*
        And with that we can now talk about the interpreter.

        In FORTH the interpreter function is often called DOCOL (I think it means "DO COLON" because
        all FORTH definitions start with a colon, as in : DOUBLE DUP + ;
```

```
        The "interpreter" (it's not really "interpreting") just needs to push the old %esi on the
        stack and set %esi to the first word in the definition.  Remember that we jumped to the
        function using JMP *(%eax)?  Well a consequence of that is that conveniently %eax contains
        the address of this codeword, so just by adding 4 to it we get the address of the first
        data word.  Finally after setting up %esi, it just does NEXT which causes that first word
        to run.
*/

/* DOCOL - the interpreter! */
        .text
        .align 4
DOCOL:
        PUSHRSP %esi            // push %esi on to the return stack
        addl $4,%eax            // %eax points to codeword, so make
        movl %eax,%esi          // %esi point to first data word
        NEXT

/*
        Just to make this absolutely clear, let's see how DOCOL works when jumping from QUADRUPLE
        into DOUBLE:

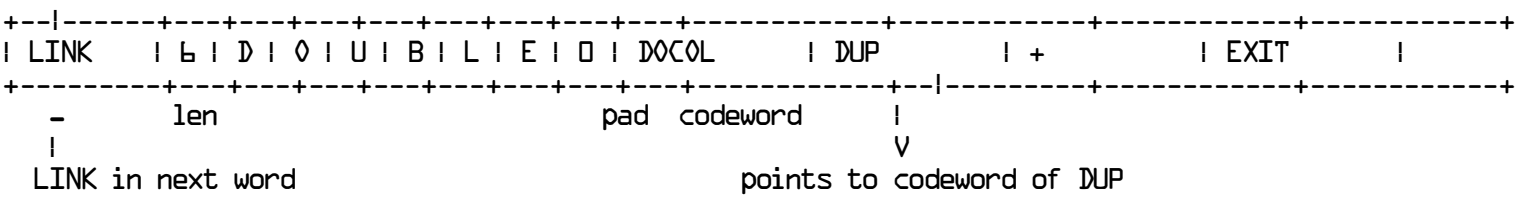
                QUADRUPLE:
                +------------------+
                | codeword         |
                +------------------+              DOUBLE:
                | addr of DOUBLE  ---------------> +------------------+
                +------------------+       %eax -> | addr of DOCOL    |
        %esi -> | addr of DOUBLE   |               +------------------+
                +------------------+               | addr of DUP      |
                | addr of EXIT     |               +------------------+
                +------------------+                | etc.             |

        First, the call to DOUBLE calls DOCOL (the codeword of DOUBLE).  DOCOL does this:  It
        pushes the old %esi on the return stack.  %eax points to the codeword of DOUBLE, so we
        just add 4 on to it to get our new %esi:

                QUADRUPLE:
                +------------------+
                | codeword         |
                +------------------+              DOUBLE:
                | addr of DOUBLE  ---------------> +------------------+
top of return   +------------------+       %eax -> | addr of DOCOL    |
stack points -> | addr of DOUBLE   |       + 4 =   +------------------+
                +------------------+       %esi -> | addr of DUP      |
                | addr of EXIT     |               +------------------+
                +------------------+                | etc.             |

        Then we do NEXT, and because of the magic of threaded code that increments %esi again
```

```
538         and calls DUP.
539
540         Well, it seems to work.
541
542         One minor point here.  Because DOCOL is the first bit of assembly actually to be defined
543         in this file (the others were just macros), and because I usually compile this code with the
544         text segment starting at address 0, DOCOL has address 0.  So if you are disassembling the
545         code and see a word with a codeword of 0, you will immediately know that the word is
546         written in FORTH (it's not an assembler primitive) and so uses DOCOL as the interpreter.
547
548         STARTING UP ------------------------------------------------------------------
549
550         Now let's get down to nuts and bolts.  When we start the program we need to set up
551         a few things like the return stack.  But as soon as we can, we want to jump into FORTH
552         code (albeit much of the "early" FORTH code will still need to be written as
553         assembly language primitives).
554
555         This is what the set up code does.  Does a tiny bit of house-keeping, sets up the
556         separate return stack (NB: Linux gives us the ordinary parameter stack already), then
557         immediately jumps to a FORTH word called QUIT.  Despite its name, QUIT doesn't quit
558         anything.  It resets some internal state and starts reading and interpreting commands.
559         (The reason it is called QUIT is because you can call QUIT from your own FORTH code
560         to "quit" your program and go back to interpreting).
561 */
562
563 /* Assembler entry point. */
564         .text
565         .globl  start
566  start:
567         cld
568         mov %esp,var_S0          // Save the initial data stack pointer in FORTH variable S0.
569         mov $return_stack_top,%ebp // Initialise the return stack.
570         call  set_up_data_segment
571
572         mov $cold_start,%esi     // Initialise interpreter.
573         NEXT                     // Run interpreter!
574
575         .section .rodata
576 cold_start:                      // High-level code without a codeword.
577         .int QUIT
578
579 /*
580         BUILT-IN WORDS ---------------------------------------------------------------
581
582         Remember our dictionary entries (headers)?  Let's bring those together with the codeword
583         and data words to see how : DOUBLE DUP + ; really looks in memory.
584
585           pointer to previous word
586            -
```



Initially we can't just write ": DOUBLE DUP + ;" (ie. that literal string) here because we don't yet have anything to read the string, break it up at spaces, parse each word, etc. etc. So instead we will have to define built-in words using the GNU assembler data constructors (like .int, .byte, .string, .ascii and so on -- look them up in the gas info page if you are unsure of them).

The long way would be:

```
        .int <link to previous word>
        .byte 6                 // len
        .ascii "DOUBLE"         // string
        .byte 0                 // padding
DOUBLE: .int DOCOL              // codeword
        .int DUP        // pointer to codeword of DUP
        .int PLUS               // pointer to codeword of +
        .int EXIT               // pointer to codeword of EXIT
```

That's going to get quite tedious rather quickly, so here I define an assembler macro so that I can just write:

```
        defword "DOUBLE",6,,DOUBLE
        .int DUP,PLUS,EXIT
```

and I'll get exactly the same effect.

Don't worry too much about the exact implementation details of this macro - it's complicated!

```
*/

/* Flags - these are discussed later. */
        .set F_IMMED,0x80
        .set F_HIDDEN,0x20
        .set F_LENMASK,0x1f      // length mask

        // Store the chain of links.
        .set link,0

        .macro defword name, namelen, flags=0, label
        .section .rodata
        .align 4
        .globl name_\label
name_\label :
```

```
        .int link               // link
        .set link,name\label
        .byte \flags+\namelen   // flags + length byte
        .ascii "\name"          // the name
        .align 4        // padding to next 4 byte boundary
        .globl \label
\label :
        .int DOCOL              // codeword - the interpreter
        // list of word pointers follow
        .endm

/*
        Similarly I want a way to write words written in assembly language.  There will be quite a few
        of these to start with because, well, everything has to start in assembly before there's
        enough "infrastructure" to be able to start writing FORTH words, but also I want to define
        some common FORTH words in assembly language for speed, even though I could write them in FORTH.

        This is what DUP looks like in memory:

          pointer to previous word
           -
           |
        +--|------+---+---+---+---+------------+
        | LINK    | 3 | D | U | P | code DUP ----------------------> points to the assembly
        +---------+---+---+---+---+------------+                     code used to write DUP,
           -       len              codeword                         which ends with NEXT.
           |
          LINK in next word

        Again, for brevity in writing the header I'm going to write an assembler macro called defcode.
        As with defword above, don't worry about the complicated details of the macro.
*/

        .macro defcode name, namelen, flags=0, label
        .section .rodata
        .align 4
        .globl name\label
name\label :
        .int link               // link
        .set link,name\label
        .byte \flags+\namelen   // flags + length byte
        .ascii "\name"          // the name
        .align 4        // padding to next 4 byte boundary
        .globl \label
\label :
        .int code\label // codeword
        .text
        //.align 4
        .globl code\label
```

```
685   code \label :                     // assembler code follows
686           .endm
687
688   /*
689           Now some easy FORTH primitives.  These are written in assembly for speed.  If you understand
690           i386 assembly language then it is worth reading these.  However if you don't understand assembly
691           you can skip the details.
692   */
693
694           defcode "DROP",4,,DROP
695           pop %eax                // drop top of stack
696           NEXT
697
698           defcode "SWAP",4,,SWAP
699           pop %eax                // swap top two elements on stack
700           pop %ebx
701           push %eax
702           push %ebx
703           NEXT
704
705           defcode "DUP",3,,DUP
706           mov (%esp),%eax         // duplicate top of stack
707           push %eax
708           NEXT
709
710           defcode "OVER",4,,OVER
711           mov 4(%esp),%eax // get the second element of stack
712           push %eax               // and push it on top
713           NEXT
714
715           defcode "ROT",3,,ROT
716           pop %eax
717           pop %ebx
718           pop %ecx
719           push %ebx
720           push %eax
721           push %ecx
722           NEXT
723
724           defcode "-ROT",4,,NROT
725           pop %eax
726           pop %ebx
727           pop %ecx
728           push %eax
729           push %ecx
730           push %ebx
731           NEXT
732
733           defcode "2DROP",5,,TWODROP // drop top two elements of stack
```

```
734             pop %eax
735             pop %eax
736             NEXT
737
738             defcode "2DUP",4,,TWODUP // duplicate top two elements of stack
739             mov (%esp),%eax
740             mov 4(%esp),%ebx
741             push %ebx
742             push %eax
743             NEXT
744
745             defcode "2SWAP",5,,TWOSWAP // swap top two pairs of elements of stack
746             pop %eax
747             pop %ebx
748             pop %ecx
749             pop %edx
750             push %ebx
751             push %eax
752             push %edx
753             push %ecx
754             NEXT
755
756             defcode "?DUP",4,,QDUP  // duplicate top of stack if non-zero
757             movl (%esp),%eax
758             test %eax,%eax
759             jz 1f
760             push %eax
761     1:      NEXT
762
763             defcode "1+",2,,INCR
764             incl (%esp)             // increment top of stack
765             NEXT
766
767             defcode "1-",2,,DECR
768             decl (%esp)             // decrement top of stack
769             NEXT
770
771             defcode "4+",2,,INCR4
772             addl $4,(%esp)          // add 4 to top of stack
773             NEXT
774
775             defcode "4-",2,,DECR4
776             subl $4,(%esp)          // subtract 4 from top of stack
777             NEXT
778
779             defcode "+",1,,ADD
780             pop %eax                // get top of stack
781             addl %eax,(%esp) // and add it to next word on stack
782             NEXT
```

```
783
784         defcode "-",1,,SUB
785         pop %eax                // get top of stack
786         subl %eax,(%esp) // and subtract it from next word on stack
787         NEXT
788
789         defcode "*",1,,MUL
790         pop %eax
791         pop %ebx
792         imull %ebx,%eax
793         push %eax               // ignore overflow
794         NEXT
795
796 /*
797         In this FORTH, only /MOD is primitive.  Later we will define the / and MOD words in
798         terms of the primitive /MOD.  The design of the i386 assembly instruction idiv which
799         leaves both quotient and remainder makes this the obvious choice.
800 */
801
802         defcode "/MOD",4,,DIVMOD
803         xor %edx,%edx
804         pop %ebx
805         pop %eax
806         idivl %ebx
807         push %edx               // push remainder
808         push %eax               // push quotient
809         NEXT
810
811 /*
812         Lots of comparison operations like =, <, >, etc..
813
814         ANS FORTH says that the comparison words should return all (binary) 1's for
815         TRUE and all 0's for FALSE.  However this is a bit of a strange convention
816         so this FORTH breaks it and returns the more normal (for C programmers ...)
817         1 meaning TRUE and 0 meaning FALSE.
818 */
819
820         defcode "=",1,,EQU      // top two words are equal?
821         pop %eax
822         pop %ebx
823         cmp %ebx,%eax
824         sete %al
825         movzbl %al,%eax
826         pushl %eax
827         NEXT
828
829         defcode "<>",2,,NEQU    // top two words are not equal?
830         pop %eax
831         pop %ebx
```

```
832             cmp %ebx,%eax
833             setne %al
834             movzbl %al,%eax
835             pushl %eax
836             NEXT
837
838             defcode "<",1,,LT
839             pop %eax
840             pop %ebx
841             cmp %eax,%ebx
842             setl %al
843             movzbl %al,%eax
844             pushl %eax
845             NEXT
846
847             defcode ">",1,,GT
848             pop %eax
849             pop %ebx
850             cmp %eax,%ebx
851             setg %al
852             movzbl %al,%eax
853             pushl %eax
854             NEXT
855
856             defcode "<=",2,,LE
857             pop %eax
858             pop %ebx
859             cmp %eax,%ebx
860             setle %al
861             movzbl %al,%eax
862             pushl %eax
863             NEXT
864
865             defcode ">=",2,,GE
866             pop %eax
867             pop %ebx
868             cmp %eax,%ebx
869             setge %al
870             movzbl %al,%eax
871             pushl %eax
872             NEXT
873
874             defcode "0=",2,,ZEQU    // top of stack equals 0?
875             pop %eax
876             test %eax,%eax
877             setz %al
878             movzbl %al,%eax
879             pushl %eax
880             NEXT
```

```
881
882         defcode "0<>",3,,ZNE0U  // top of stack not 0?
883         pop %eax
884         test %eax,%eax
885         setnz %al
886         movzbl %al,%eax
887         pushl %eax
888         NEXT
889
890         defcode "0<",2,,ZLT     // comparisons with 0
891         pop %eax
892         test %eax,%eax
893         setl %al
894         movzbl %al,%eax
895         pushl %eax
896         NEXT
897
898         defcode "0>",2,,ZGT
899         pop %eax
900         test %eax,%eax
901         setg %al
902         movzbl %al,%eax
903         pushl %eax
904         NEXT
905
906         defcode "0<=",3,,ZLE
907         pop %eax
908         test %eax,%eax
909         setle %al
910         movzbl %al,%eax
911         pushl %eax
912         NEXT
913
914         defcode "0>=",3,,ZGE
915         pop %eax
916         test %eax,%eax
917         setge %al
918         movzbl %al,%eax
919         pushl %eax
920         NEXT
921
922         defcode "AND",3,,AND    // bitwise AND
923         pop %eax
924         andl %eax,(%esp)
925         NEXT
926
927         defcode "OR",2,,OR      // bitwise OR
928         pop %eax
929         orl %eax,(%esp)
```

```
        NEXT

        defcode "XOR",3,,XOR    // bitwise XOR
        pop %eax
        xorl %eax,(%esp)
        NEXT

        defcode "INVERT",6,,INVERT // this is the FORTH bitwise "NOT" function (cf. NEGATE and NOT)
        notl (%esp)
        NEXT

/*
        RETURNING FROM FORTH WORDS ---------------------------------------------------------------------

        Time to talk about what happens when we EXIT a function.  In this diagram QUADRUPLE has called
        DOUBLE, and DOUBLE is about to exit (look at where %esi is pointing):

                QUADRUPLE
                +------------------+
                | codeword         |
                +------------------+              DOUBLE
                | addr of DOUBLE  ---------------> +------------------+
                +------------------+               | codeword         |
                | addr of DOUBLE   |               +------------------+
                +------------------+               | addr of DUP      |
                | addr of EXIT     |               +------------------+
                +------------------+               | addr of +        |
                                                   +------------------+
                                        %esi -> | addr of EXIT     |
                                                   +------------------+

        What happens when the + function does NEXT?  Well, the following code is executed.
*/

        defcode "EXIT",4,,EXIT
        POPRSP %esi             // pop return stack into %esi
        NEXT

/*
        EXIT gets the old %esi which we saved from before on the return stack, and puts it in %esi.
        So after this (but just before NEXT) we get:

                QUADRUPLE
                +------------------+
                | codeword         |
                +------------------+              DOUBLE
                | addr of DOUBLE  ---------------> +------------------+
                +------------------+               | codeword         |
        %esi -> | addr of DOUBLE   |               +------------------+
```

```
                +------------------+              | addr of DUP      |
                | addr of EXIT     |              +------------------+
                +------------------+              | addr of +        |
                                                  +------------------+
                                                  | addr of EXIT     |
                                                  +------------------+
```

And NEXT just completes the job by, well, in this case just by calling DOUBLE again :-)

LITERALS ----------

The final point I "glossed over" before was how to deal with functions that do anything apart from calling other functions. For example, suppose that DOUBLE was defined like this:

```
: DOUBLE 2 * ;
```

It does the same thing, but how do we compile it since it contains the literal 2? One way would be to have a function called "2" (which you'd have to write **in** assembler), but you'd need a function for every single literal that you wanted to use.

FORTH solves this by compiling the function using a special word called LIT:

```
+---------------------------+-------+-------+-------+-------+-------+
| (usual header of DOUBLE)  | DOCOL | LIT   | 2     | *     | EXIT  |
+---------------------------+-------+-------+-------+-------+-------+
```

LIT is executed in the normal way, but what it does next is definitely not normal. It looks at %esi (which now points to the number 2), grabs it, pushes it on the stack, then manipulates %esi in order to skip the number as if it had never been there.

What's neat is that the whole grab/manipulate can be done using a single byte single i386 instruction, our old friend LODSL. Rather than me drawing more ASCII-art diagrams, see **if** you can find out how LIT works:

```
*/

        defcode "LIT",3,,LIT
        // %esi points to the next command, but in this case it points to the next
        // literal 32 bit integer.  Get that literal into %eax and increment %esi.
        // On x86, it's a convenient single byte instruction!  (cf. NEXT macro)
        lodsl
        push %eax               // push the literal number on to stack
        NEXT

/*
```

MEMORY ----------

An important point about FORTH is that it gives you direct access to the lowest levels of the machine. Manipulating memory directly is done frequently in FORTH, and these are the primitive words for doing it.

```
1028  */
1029
1030          defcode "!",1,,STORE
1031          pop %ebx                // address to store at
1032          pop %eax                // data to store there
1033          mov %eax,(%ebx)         // store it
1034          NEXT
1035
1036          defcode "@",1,,FETCH
1037          pop %ebx                // address to fetch
1038          mov (%ebx),%eax         // fetch it
1039          push %eax               // push value onto stack
1040          NEXT
1041
1042          defcode "+!",2,,ADDSTORE
1043          pop %ebx                // address
1044          pop %eax                // the amount to add
1045          addl %eax,(%ebx) // add it
1046          NEXT
1047
1048          defcode "-!",2,,SUBSTORE
1049          pop %ebx                // address
1050          pop %eax                // the amount to subtract
1051          subl %eax,(%ebx) // add it
1052          NEXT
1053
1054  /*
1055          ! and @ (STORE and FETCH) store 32-bit words.  It's also useful to be able to read and write bytes
1056          so we also define standard words C@ and C!.
1057
1058          Byte-oriented operations only work on architectures which permit them (i386 is one of those).
1059   */
1060
1061          defcode "C!",2,,STOREBYTE
1062          pop %ebx                // address to store at
1063          pop %eax                // data to store there
1064          movb %al,(%ebx)         // store it
1065          NEXT
1066
1067          defcode "C@",2,,FETCHBYTE
1068          pop %ebx                // address to fetch
1069          xor %eax,%eax
1070          movb (%ebx),%al         // fetch it
1071          push %eax               // push value onto stack
1072          NEXT
1073
1074  /* C@C! is a useful byte copy primitive. */
1075          defcode "C@C!",4,,CCOPY
1076          movl 4(%esp),%ebx       // source address
```

```
        movb (%ebx),%al         // get source character
        pop %edi                // destination address
        stosb                   // copy to destination
        push %edi               // increment destination address
        incl 4(%esp)            // increment source address
        NEXT

/* and CMOVE is a block copy operation. */
        defcode "CMOVE",5,,CMOVE
        mov %esi,%edx           // preserve %esi
        pop %ecx                // length
        pop %edi                // destination address
        pop %esi                // source address
        rep movsb               // copy source to destination
        mov %edx,%esi           // restore %esi
        NEXT

/*
        BUILT-IN VARIABLES ----------------------------------------------------------------

        These are some built-in variables and related standard FORTH words.  Of these, the only one that we
        have discussed so far was LATEST, which points to the last (most recently defined) word in the
        FORTH dictionary.  LATEST is also a FORTH word which pushes the address of LATEST (the variable)
        on to the stack, so you can read or write it using @ and ! operators.  For example, to print
        the current value of LATEST (and this can apply to any FORTH variable) you would do:

        LATEST @ . CR

        To make defining variables shorter, I'm using a macro called defvar, similar to defword and
        defcode above.  (In fact the defvar macro uses defcode to do the dictionary header).
*/

        .macro defvar name, namelen, flags=0, label, initial=0
        defcode \name,\namelen,\flags,\label
        push $var \name
        NEXT
        .data
        .align 4
var \name :
        .int \initial
        .endm

/*
        The built-in variables are:

        STATE           Is the interpreter executing code (0) or compiling a word (non-zero)?
        LATEST          Points to the latest (most recently defined) word in the dictionary.
        HERE            Points to the next free byte of memory.  When compiling, compiled words go here.
        S0              Stores the address of the top of the parameter stack.
```

```
        BASE            The current base for printing and reading numbers.

*/
        defvar "STATE",5,,STATE
        defvar "HERE",4,,HERE
        defvar "LATEST",6,,LATEST,name SYSCALL0 // SYSCALL0 must be last in built-in dictionary
        defvar "S0",2,,SZ
        defvar "BASE",4,,BASE,10

/*
        BUILT-IN CONSTANTS -----------------------------------------------------------

        It's also useful to expose a few constants to FORTH.  When the word is executed it pushes a
        constant value on the stack.

        The built-in constants are:

        VERSION         Is the current version of this FORTH.
        R0              The address of the top of the return stack.
        DOCOL           Pointer to DOCOL.
        F_IMMED         The IMMEDIATE flag's actual value.
        F_HIDDEN        The HIDDEN flag's actual value.
        F_LENMASK       The length mask in the flags/len byte.

        SYS_*           and the numeric codes of various Linux syscalls (from <asm/unistd.h>)
*/

//#include <asm-i386/unistd.h>  // you might need this instead
#include <asm/unistd.h>

        .macro defconst name, namelen, flags=0, label, value
        defcode \name,\namelen,\flags,\label
        push $\value
        NEXT
        .endm

        defconst "VERSION",7,,VERSION,JONES_VERSION
        defconst "R0",2,,RZ,return_stack_top
        defconst "DOCOL",5,, DOCOL,DOCOL
        defconst "F_IMMED",7,, F_IMMED,F_IMMED
        defconst "F_HIDDEN",8,, F_HIDDEN,F_HIDDEN
        defconst "F_LENMASK",9,, F_LENMASK,F_LENMASK

        defconst "SYS_EXIT",8,,SYS_EXIT, __NR_exit
        defconst "SYS_OPEN",8,,SYS_OPEN, __NR_open
        defconst "SYS_CLOSE",9,,SYS_CLOSE, __NR_close
        defconst "SYS_READ",8,,SYS_READ, __NR_read
        defconst "SYS_WRITE",9,,SYS_WRITE, __NR_write
        defconst "SYS_CREAT",9,,SYS_CREAT, __NR_creat
```

```
1175          defconst "SYS_BRK",7,,SYS_BRK, __NR_brk
1176
1177          defconst "O_RDONLY",8,, 0 RDONLY,0
1178          defconst "O_WRONLY",8,, 0 WRONLY,1
1179          defconst "O_RDWR",6,, 0 RDWR,2
1180          defconst "O_CREAT",7,, 0 CREAT,0100
1181          defconst "O_EXCL",6,, 0 EXCL,0200
1182          defconst "O_TRUNC",7,, 0 TRUNC,01000
1183          defconst "O_APPEND",8,, 0 APPEND,02000
1184          defconst "O_NONBLOCK",10,, 0 NONBLOCK,04000
1185
1186    /*
1187          RETURN STACK ------------------------------------------------------------------
1188
1189          These words allow you to access the return stack.  Recall that the register %ebp always points to
1190          the top of the return stack.
1191    */
1192
1193          defcode ">R",2,,TOR
1194          pop %eax                // pop parameter stack into %eax
1195          PUSHRSP %eax            // push it on to the return stack
1196          NEXT
1197
1198          defcode "R>",2,,FROMR
1199          POPRSP %eax             // pop return stack on to %eax
1200          push %eax               // and push on to parameter stack
1201          NEXT
1202
1203          defcode "RSP@",4,,RSPFETCH
1204          push %ebp
1205          NEXT
1206
1207          defcode "RSP!",4,,RSPSTORE
1208          pop %ebp
1209          NEXT
1210
1211          defcode "RDROP",5,,RDROP
1212          addl $4,%ebp            // pop return stack and throw away
1213          NEXT
1214
1215    /*
1216          PARAMETER (DATA) STACK --------------------------------------------------------
1217
1218          These functions allow you to manipulate the parameter stack.  Recall that Linux sets up the parameter
1219          stack for us, and it is accessed through %esp.
1220    */
1221
1222          defcode "DSP@",4,,DSPFETCH
1223          mov %esp,%eax
```

```
        push %eax
        NEXT

        defcode "DSP!",4,,DSPSTORE
        pop %esp
        NEXT

/*
        INPUT AND OUTPUT ----------------------------------------------------------------------

        These are our first really meaty/complicated FORTH primitives.  I have chosen to write them in
        assembler, but surprisingly in "real" FORTH implementations these are often written in terms
        of more fundamental FORTH primitives.  I chose to avoid that because I think that just obscures
        the implementation.  After all, you may not understand assembler but you can just think of it
        as an opaque block of code that does what it says.

        Let's discuss input first.

        The FORTH word KEY reads the next byte from stdin (and pushes it on the parameter stack).
        So if KEY is called and someone hits the space key, then the number 32 (ASCII code of space)
        is pushed on the stack.

        In FORTH there is no distinction between reading code and reading input.  We might be reading
        and compiling code, we might be reading words to execute, we might be asking for the user
        to type their name -- ultimately it all comes in through KEY.

        The implementation of KEY uses an input buffer of a certain size (defined at the end of this
        file).  It calls the Linux read(2) system call to fill this buffer and tracks its position
        in the buffer using a couple of variables, and if it runs out of input buffer then it refills
        it automatically.  The other thing that KEY does is if it detects that stdin has closed, it
        exits the program, which is why when you hit ^D the FORTH system cleanly exits.

    buffer                          bufftop
       |                               |
       V                               V
       +-------------------------------+---------------------------------------+
       | INPUT READ FROM STDIN ..... | unused part of the buffer           |
       +-------------------------------+---------------------------------------+
                        -
                        |
                  currkey (next character to read)

       <---------------------- BUFFER SIZE (4096 bytes) ---------------------->
*/

        defcode "KEY",3,,KEY
        call  KEY
        push %eax               // push return value on stack
        NEXT
```

```
KEY:
        mov (currkey),%ebx
        cmp (bufftop),%ebx
        jge 1f                  // exhausted the input buffer?
        xor %eax,%eax
        mov (%ebx),%al          // get next key from input buffer
        inc %ebx
        mov %ebx,(currkey)      // increment currkey
        ret

1:      // Out of input; use read(2) to fetch more input from stdin.
        xor %ebx,%ebx           // 1st param: stdin
        mov $buffer,%ecx        // 2nd param: buffer
        mov %ecx,currkey
        mov $BUFFER_SIZE,%edx   // 3rd param: max length
        mov $__NR_read,%eax     // syscall: read
        int $0x80
        test %eax,%eax          // If %eax <= 0, then exit.
        jbe 2f
        addl %eax,%ecx          // buffer+%eax = bufftop
        mov %ecx,bufftop
        jmp  KEY

2:      // Error or end of input: exit the program.
        xor %ebx,%ebx
        mov $__NR_exit,%eax     // syscall: exit
        int $0x80

        .data
        .align 4
currkey:
        .int buffer             // Current place in input buffer (next character to read).
bufftop:
        .int buffer             // Last valid data in input buffer + 1.

/*
        By contrast, output is much simpler.  The FORTH word EMIT writes out a single byte to stdout.
        This implementation just uses the write system call.  No attempt is made to buffer output, but
        it would be a good exercise to add it.
*/

        defcode "EMIT",4,,EMIT
        pop %eax
        call  EMIT
        NEXT
EMIT:
        mov $1,%ebx             // 1st param: stdout

        // write needs the address of the byte to write
```

```
1322            mov %al,emit_scratch
1323            mov $emit_scratch,%ecx  // 2nd param: address
1324
1325            mov $1,%edx             // 3rd param: nbytes = 1
1326
1327            mov $ __NR_write,%eax   // write syscall
1328            int $0x80
1329            ret
1330
1331            .data                   // NB: easier to fit in the .data section
1332    emit_scratch:
1333            .space 1        // scratch used by EMIT
1334
1335    /*
1336            Back to input, WORD is a FORTH word which reads the next full word of input.
1337
1338            What it does in detail is that it first skips any blanks (spaces, tabs, newlines and so on).
1339            Then it calls KEY to read characters into an internal buffer until it hits a blank.  Then it
1340            calculates the length of the word it read and returns the address and the length as
1341            two words on the stack (with the length at the top of stack).
1342
1343            Notice that WORD has a single internal buffer which it overwrites each time (rather like
1344            a static C string).  Also notice that WORD's internal buffer is just 32 bytes long and
1345            there is NO checking for overflow.  31 bytes happens to be the maximum length of a
1346            FORTH word that we support, and that is what WORD is used for: to read FORTH words when
1347            we are compiling and executing code.  The returned strings are not NUL-terminated.
1348
1349            Start address+length is the normal way to represent strings in FORTH (not ending in an
1350            ASCII NUL character as in C), and so FORTH strings can contain any character including NULs
1351            and can be any length.
1352
1353            WORD is not suitable for just reading strings (eg. user input) because of all the above
1354            peculiarities and limitations.
1355
1356            Note that when executing, you'll see:
1357            WORD FOO
1358            which puts "FOO" and length 3 on the stack, but when compiling:
1359            : BAR WORD FOO ;
1360            is an error (or at least it doesn't do what you might expect).  Later we'll talk about compiling
1361            and immediate mode, and you'll understand why.
1362    */
1363
1364            defcode "WORD",4,,WORD
1365            call  WORD
1366            push %edi               // push base address
1367            push %ecx               // push length
1368            NEXT
1369
1370     WORD:
```

```
1371            /* Search for first non-blank character.  Also skip \ comments. */
1372    1:
1373            call  KEY               // get next key, returned in %eax
1374            cmpb $'\\',%al          // start of a comment?
1375            je 3f                   // if so, skip the comment
1376            cmpb $' ',%al
1377            jbe 1b                  // if so, keep looking
1378
1379            /* Search for the end of the word, storing chars as we go. */
1380            mov $word_buffer,%edi   // pointer to return buffer
1381    2:
1382            stosb                   // add character to return buffer
1383            call  KEY               // get next key, returned in %al
1384            cmpb $' ',%al           // is blank?
1385            ja 2b                   // if not, keep looping
1386
1387            /* Return the word (well, the static buffer) and length. */
1388            sub $word_buffer,%edi
1389            mov %edi,%ecx           // return length of the word
1390            mov $word_buffer,%edi   // return address of the word
1391            ret
1392
1393            /* Code to skip \ comments to end of the current line. */
1394    3:
1395            call  KEY
1396            cmpb $'\n',%al          // end of line yet?
1397            jne 3b
1398            jmp 1b
1399
1400            .data                   // NB: easier to fit in the .data section
1401            // A static buffer where WORD returns.  Subsequent calls
1402            // overwrite this buffer.  Maximum word length is 32 chars.
1403    word_buffer:
1404            .space 32
1405
1406    /*
1407            As well as reading in words we'll need to read in numbers and for that we are using a function
1408            called NUMBER.  This parses a numeric string such as one returned by WORD and pushes the
1409            number on the parameter stack.
1410
1411            The function uses the variable BASE as the base (radix) for conversion, so for example if
1412            BASE is 2 then we expect a binary number.  Normally BASE is 10.
1413
1414            If the word starts with a '-' character then the returned value is negative.
1415
1416            If the string can't be parsed as a number (or contains characters outside the current BASE)
1417            then we need to return an error indication.  So NUMBER actually returns two items on the stack.
1418            At the top of stack we return the number of unconverted characters (ie. if 0 then all characters
1419            were converted, so there is no error).  Second from top of stack is the parsed number or a
```

```
1420            partial value if there was an error.
1421    */
1422            defcode "NUMBER",6,,NUMBER
1423            pop %ecx                // length of string
1424            pop %edi                // start address of string
1425            call  NUMBER
1426            push %eax               // parsed number
1427            push %ecx               // number of unparsed characters (0 = no error)
1428            NEXT
1429
1430     NUMBER:
1431            xor %eax,%eax
1432            xor %ebx,%ebx
1433
1434            test %ecx,%ecx          // trying to parse a zero-length string is an error, but will return 0.
1435            jz 5f
1436
1437            movl var_BASE,%edx      // get BASE (in %dl)
1438
1439            // Check if first character is '-'.
1440            movb (%edi),%bl         // %bl = first character in string
1441            inc %edi
1442            push %eax               // push 0 on stack
1443            cmpb $'-',%bl           // negative number?
1444            jnz 2f
1445            pop %eax
1446            push %ebx               // push <> 0 on stack, indicating negative
1447            dec %ecx
1448            jnz 1f
1449            pop %ebx                // error: string is only '-'.
1450            movl $1,%ecx
1451            ret
1452
1453            // Loop reading digits.
1454    1:      imull %edx,%eax         // %eax *= BASE
1455            movb (%edi),%bl         // %bl = next character in string
1456            inc %edi
1457
1458            // Convert 0-9, A-Z to a number 0-35.
1459    2:      subb $'0',%bl           // < '0'?
1460            jb 4f
1461            cmp $10,%bl             // <= '9'?
1462            jb 3f
1463            subb $17,%bl            // < 'A'? (17 is 'A'-'0')
1464            jb 4f
1465            addb $10,%bl
1466
1467    3:      cmp %dl,%bl             // >= BASE?
1468            jge 4f
```

```
        // OK, so add it to %eax and loop.
        add %ebx,%eax
        dec %ecx
        jnz 1b

        // Negate the result if first character was '-' (saved on the stack).
4:      pop %ebx
        test %ebx,%ebx
        jz 5f
        neg %eax

5:      ret

/*
        DICTIONARY LOOK UPS ----------------------------------------------------------------------

        We're building up to our prelude on how FORTH code is compiled, but first we need yet more infrastructure.

        The FORTH word FIND takes a string (a word as parsed by WORD -- see above) and looks it up in the
        dictionary.  What it actually returns is the address of the dictionary header, if it finds it,
        or 0 if it didn't.

        So if DOUBLE is defined in the dictionary, then WORD DOUBLE FIND returns the following pointer:

    pointer to this
        |
        |
        V
        +---------+---+---+---+---+---+---+---+---+-----------+-----------+-----------+-----------+
        | LINK    | 6 | D | O | U | B | L | E | 0 | DOCOL     | DUP       | +         | EXIT      |
        +---------+---+---+---+---+---+---+---+---+-----------+-----------+-----------+-----------+

        See also >CFA and >DFA.

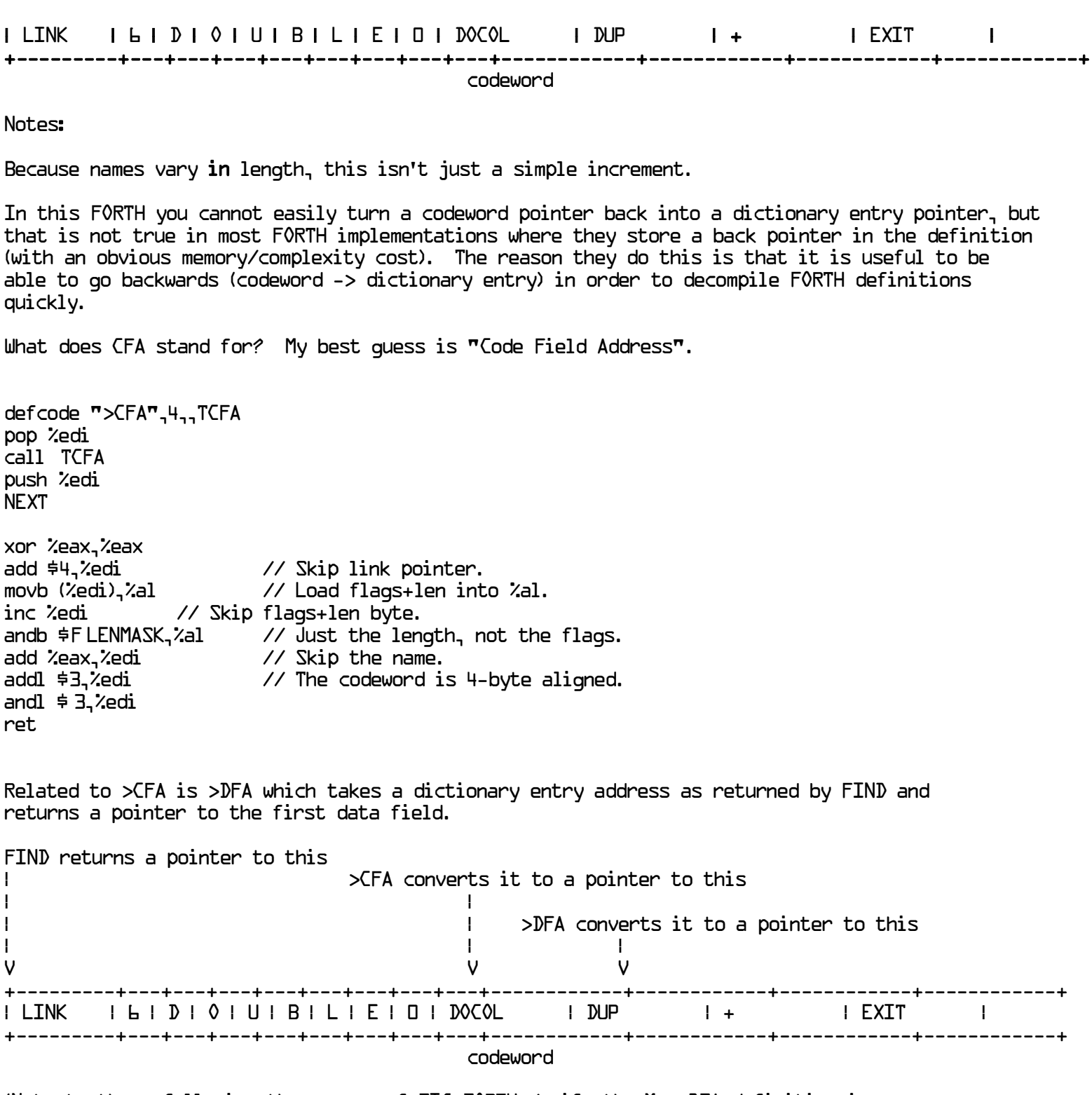
        FIND doesn't find dictionary entries which are flagged as HIDDEN.  See below for why.
*/

        defcode "FIND",4,,FIND
        pop %ecx                // %ecx = length
        pop %edi                // %edi = address
        call  FIND
        push %eax               // %eax = address of dictionary entry (or NULL)
        NEXT

 FIND:
        push %esi               // Save %esi so we can use it in string comparison.

        // Now we start searching backwards through the dictionary for this word.
```

```
1518            mov var_LATEST,%edx     // LATEST points to name header of the latest word in the dictionary
1519    1:      test %edx,%edx          // NULL pointer?  (end of the linked list)
1520            je 4f
1521
1522            // Compare the length expected and the length of the word.
1523            // Note that if the F_HIDDEN flag is set on the word, then by a bit of trickery
1524            // this won't pick the word (the length will appear to be wrong).
1525            xor %eax,%eax
1526            movb 4(%edx),%al // %al = flags+length field
1527            andb $(F_HIDDEN|F_LENMASK),%al // %al = name length
1528            cmpb %cl,%al            // Length is the same?
1529            jne 2f
1530
1531            // Compare the strings in detail.
1532            push %ecx               // Save the length
1533            push %edi               // Save the address (repe cmpsb will move this pointer)
1534            lea 5(%edx),%esi // Dictionary string we are checking against.
1535            repe cmpsb              // Compare the strings.
1536            pop %edi
1537            pop %ecx
1538            jne 2f                  // Not the same.
1539
1540            // The strings are the same - return the header pointer in %eax
1541            pop %esi
1542            mov %edx,%eax
1543            ret
1544
1545    2:      mov (%edx),%edx         // Move back through the link field to the previous word
1546            jmp 1b                  // .. and loop.
1547
1548    4:      // Not found.
1549            pop %esi
1550            xor %eax,%eax           // Return zero to indicate not found.
1551            ret
1552
1553    /*
1554            FIND returns the dictionary pointer, but when compiling we need the codeword pointer (recall
1555            that FORTH definitions are compiled into lists of codeword pointers).  The standard FORTH
1556            word >CFA turns a dictionary pointer into a codeword pointer.
1557
1558            The example below shows the result of:
1559
1560                    WORD DOUBLE FIND >CFA
1561
1562            FIND returns a pointer to this
1563            |                               >CFA converts it to a pointer to this
1564            |                                       |
1565            V                                       V
1566            +---------+---+---+---+---+---+---+---+---+-----------+-----------+-----------+-----------+
```



Notes:

Because names vary **in** length, this isn't just a simple increment.

In this FORTH you cannot easily turn a codeword pointer back into a dictionary entry pointer, but that is not true in most FORTH implementations where they store a back pointer in the definition (with an obvious memory/complexity cost). The reason they do this is that it is useful to be able to go backwards (codeword -> dictionary entry) in order to decompile FORTH definitions quickly.

What does CFA stand for? My best guess is "Code Field Address".

*/

```
        defcode ">CFA",4,,TCFA
        pop %edi
        call  TCFA
        push %edi
        NEXT
 TCFA:
        xor %eax,%eax
        add $4,%edi             // Skip link pointer.
        movb (%edi),%al         // Load flags+len into %al.
        inc %edi        // Skip flags+len byte.
        andb $F LENMASK,%al     // Just the length, not the flags.
        add %eax,%edi           // Skip the name.
        addl $3,%edi            // The codeword is 4-byte aligned.
        andl $ 3,%edi
        ret
```

/*

Related to >CFA is >DFA which takes a dictionary entry address as returned by FIND and returns a pointer to the first data field.

FIND returns a pointer to this

>CFA converts it to a pointer to this

>DFA converts it to a pointer to this

(Note to those following the source of FIG-FORTH / ciforth: My >DFA definition is

```
        different from theirs, because they have an extra indirection).

        You can see that >DFA is easily defined in FORTH just by adding 4 to the result of >CFA.
*/

        defword ">DFA",4,,TDFA
        .int TCFA               // >CFA         (get code field address)
        .int INCR4              // 4+           (add 4 to it to get to next word)
        .int EXIT               // EXIT         (return from FORTH word)

/*
        COMPILING ---------------------------------------------------------------

        Now we'll talk about how FORTH compiles words.  Recall that a word definition looks like this:

                : DOUBLE DUP + ;

        and we have to turn this into:

          pointer to previous word
           -
           |
        +--|------+---+---+---+---+---+---+---+---+------------+------------+------------+------------+
        | LINK    | 6 | D | O | U | B | L | E | 0 | DOCOL      | DUP        | +          | EXIT       |
        +---------+---+---+---+---+---+---+---+---+------------+--|---------+------------+------------+
           -       len                          pad  codeword      |
           |                                                       V
          LATEST points here                              points to codeword of DUP

        There are several problems to solve.  Where to put the new word?  How do we read words?  How
        do we define the words : (COLON) and ; (SEMICOLON)?

        FORTH solves this rather elegantly and as you might expect in a very low-level way which
        allows you to change how the compiler works on your own code.

        FORTH has an INTERPRET function (a true interpreter this time, not DOCOL) which runs in a
        loop, reading words (using WORD), looking them up (using FIND), turning them into codeword
        pointers (using >CFA) and deciding what to do with them.

        What it does depends on the mode of the interpreter (in variable STATE).

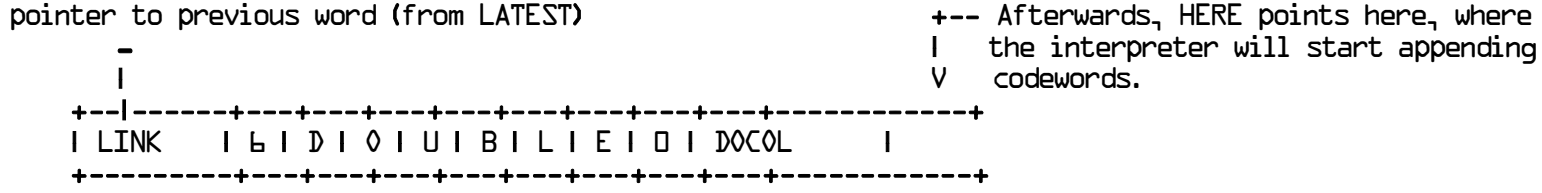
        When STATE is zero, the interpreter just runs each word as it looks them up.  This is known as
        immediate mode.

        The interesting stuff happens when STATE is non-zero -- compiling mode.  In this mode the
        interpreter appends the codeword pointer to user memory (the HERE variable points to the next
        free byte of user memory -- see DATA SEGMENT section below).

        So you may be able to see how we could define : (COLON).  The general plan is:
```

(1) Use WORD to read the name of the **function** being defined.

(2) Construct the dictionary entry -- just the header part -- **in** user memory:



(3) Set LATEST to point to the newly defined word, ...

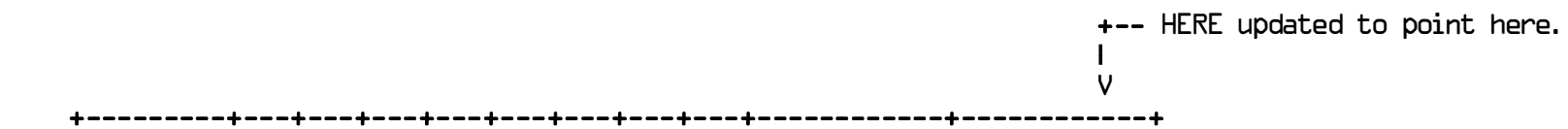
(4) .. and most importantly leave HERE pointing just after the new codeword. This is where the interpreter will append codewords.

(5) Set STATE to 1. This goes into compile mode so the interpreter starts appending codewords to our partially-formed header.

After **:** has run, our input is here:

```
: DOUBLE DUP + ;
         -
         |
        Next byte returned by KEY will be the 'D' character of DUP
```

so the interpreter (now it's in compile mode, so I guess it's really the compiler) reads "DUP", looks it up **in** the dictionary, gets its codeword pointer, and appends it:



Next we read +, get the codeword pointer, and append it:



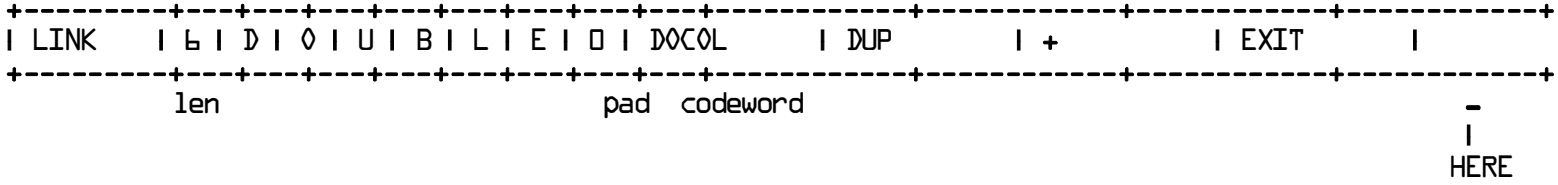
The issue is what happens next. Obviously what we don't want to happen is that we read ";" and compile it and go on compiling everything afterwards.

At this point, FORTH uses a trick. Remember the length byte in the dictionary definition isn't just a plain length byte, but can also contain flags. One flag is called the IMMEDIATE flag (F_IMMED in this code). If a word in the dictionary is flagged as IMMEDIATE then the interpreter runs it immediately even if it's in compile mode.

This is how the word ; (SEMICOLON) works -- as a word flagged in the dictionary as IMMEDIATE.

And all it does is append the codeword for EXIT on to the current definition and switch back to immediate mode (set STATE back to 0). Shortly we'll see the actual definition of ; and we'll see that it's really a very simple definition, declared IMMEDIATE.

After the interpreter reads ; and executes it 'immediately', we get this:

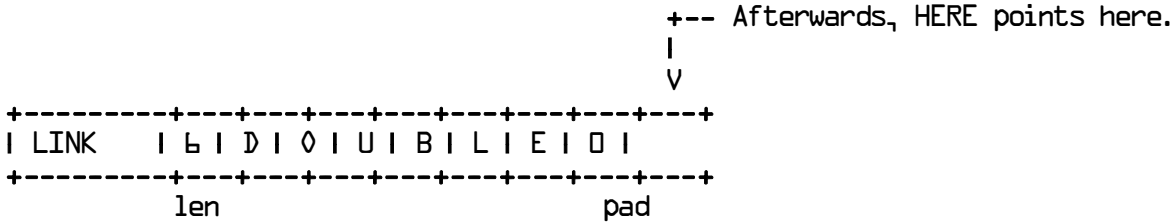


STATE is set to 0.

And that's it, job done, our new definition is compiled, and we're back in immediate mode just reading and executing words, perhaps including a call to test our new word DOUBLE.

The only last wrinkle in this is that **while** our word was being compiled, it was in a half-finished state. We certainly wouldn't want DOUBLE to be called somehow during this time. There are several ways to stop this from happening, but in FORTH what we do is flag the word with the HIDDEN flag (F_HIDDEN in this code) just while it is being compiled. This prevents FIND from finding it, and thus in theory stops any chance of it being called.

The above explains how compiling, : (COLON) and ; (SEMICOLON) works and in a moment I'm going to define them. The : (COLON) **function** can be made a little bit more general by writing it in two parts. The first part, called CREATE, makes just the header:



and the second part, the actual definition of : (COLON), calls CREATE and appends the DOCOL codeword, so leaving:

```
                                                                +-- Afterwards, HERE points here.
                                                                |
                                                                V
        +---------+---+---+---+---+---+---+---+---+------------+
        | LINK    | 6 | D | 0 | U | B | L | E | 0 | DOCOL      |
        +---------+---+---+---+---+---+---+---+---+------------+
                   len                           pad  codeword
```

CREATE is a standard FORTH word and the advantage of this split is that we can reuse it to create other types of words (not just ones which contain code, but words which contain variables, constants and other data).

```
*/

        defcode "CREATE",6,,CREATE

        // Get the name length and address.
        pop %ecx                // %ecx = length
        pop %ebx                // %ebx = address of name

        // Link pointer.
        movl var HERE,%edi      // %edi is the address of the header
        movl var LATEST,%eax    // Get link pointer
        stosl                   // and store it in the header.

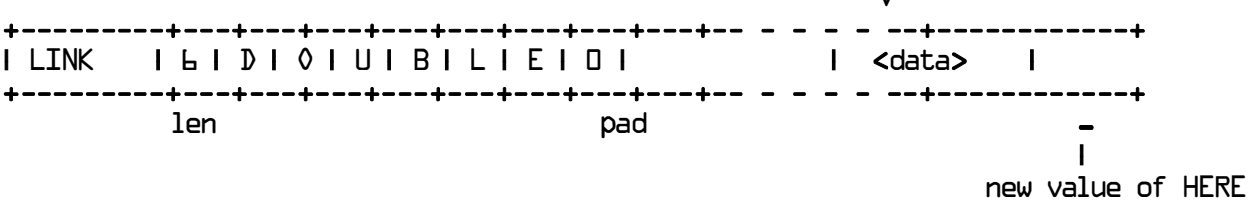
        // Length byte and the word itself.
        mov %cl,%al             // Get the length.
        stosb                   // Store the length/flags byte.
        push %esi
        mov %ebx,%esi           // %esi = word
        rep movsb               // Copy the word
        pop %esi
        addl $3,%edi            // Align to next 4 byte boundary.
        andl $ 3,%edi

        // Update LATEST and HERE.
        movl var HERE,%eax
        movl %eax,var LATEST
        movl %edi,var HERE
        NEXT

/*
        Because I want to define : (COLON) in FORTH, not assembler, we need a few more FORTH words
        to use.

        The first is , (COMMA) which is a standard FORTH word which appends a 32 bit integer to the user
        memory pointed to by HERE, and adds 4 to HERE.  So the action of , (COMMA) is:

                                                        previous value of HERE
                                                                |
```



and <data> is whatever 32 bit integer was at the top of the stack.

, (COMMA) is quite a fundamental operation when compiling. It is used to append codewords to the current word that is being compiled.

```
*/

        defcode ",",1,,COMMA
        pop %eax                // Code pointer to store.
        call  COMMA
        NEXT
 COMMA:
        movl var HERE,%edi      // HERE
        stosl                   // Store it.
        movl %edi,var HERE      // Update HERE (incremented)
        ret

/*
```

Our definitions of : (COLON) and ; (SEMICOLON) will need to switch to and from compile mode.

Immediate mode vs. compile mode is stored in the global variable STATE, and by updating this variable we can switch between the two modes.

For various reasons which may become apparent later, FORTH defines two standard words called [and] (LBRAC and RBRAC) which switch between modes:

Word	Assembler	Action	Effect
[	LBRAC	STATE := 0	Switch to immediate mode.
]	RBRAC	STATE := 1	Switch to compile mode.

[(LBRAC) is an IMMEDIATE word. The reason is as follows: If we are in compile mode and the interpreter saw [then it would compile it rather than running it. We would never be able to switch back to immediate mode! So we flag the word as IMMEDIATE so that even in compile mode the word runs immediately, switching us back to immediate mode.

```
*/

        defcode "[",1,F_IMMED,LBRAC
        xor %eax,%eax
        movl %eax,var STATE     // Set STATE to 0.
        NEXT

        defcode "]",1,,RBRAC
```

```
        movl $1,var_STATE       // Set STATE to 1.
        NEXT

/*
        Now we can define : (COLON) using CREATE.  It just calls CREATE, appends DOCOL (the codeword), sets
        the word HIDDEN and goes into compile mode.
*/

        defword ":",1,,COLON
        .int WORD               // Get the name of the new word
        .int CREATE             // CREATE the dictionary entry / header
        .int LIT, DOCOL, COMMA  // Append DOCOL  (the codeword).
        .int LATEST, FETCH, HIDDEN // Make the word hidden (see below for definition).
        .int RBRAC              // Go into compile mode.
        .int EXIT               // Return from the function.

/*
        ; (SEMICOLON) is also elegantly simple.  Notice the F_IMMED flag.
*/

        defword ";",1,F_IMMED,SEMICOLON
        .int LIT, EXIT, COMMA   // Append EXIT (so the word will return).
        .int LATEST, FETCH, HIDDEN // Toggle hidden flag -- unhide the word (see below for definition).
        .int LBRAC              // Go back to IMMEDIATE mode.
        .int EXIT               // Return from the function.

/*
        EXTENDING THE COMPILER ------------------------------------------------------------------

        Words flagged with IMMEDIATE (F_IMMED) aren't just for the FORTH compiler to use.  You can define
        your own IMMEDIATE words too, and this is a crucial aspect when extending basic FORTH, because
        it allows you in effect to extend the compiler itself.  Does gcc let you do that?

        Standard FORTH words like IF, WHILE, ." and so on are all written as extensions to the basic
        compiler, and are all IMMEDIATE words.

        The IMMEDIATE word toggles the F_IMMED (IMMEDIATE flag) on the most recently defined word,
        or on the current word if you call it in the middle of a definition.

        Typical usage is:

        : MYIMMEDWORD IMMEDIATE
                ...definition...
        ;

        but some FORTH programmers write this instead:

        : MYIMMEDWORD
                ...definition...
```

```
1910            ; IMMEDIATE
1911
1912            The two usages are equivalent, to a first approximation.
1913    */
1914
1915            defcode "IMMEDIATE",9,F_IMMED,IMMEDIATE
1916            movl var_LATEST,%edi     // LATEST word.
1917            addl $4,%edi             // Point to name/flags byte.
1918            xorb $F_IMMED,(%edi)     // Toggle the IMMED bit.
1919            NEXT
1920
1921    /*
1922            'addr HIDDEN' toggles the hidden flag (F_HIDDEN) of the word defined at addr.  To hide the
1923            most recently defined word (used above in : and ; definitions) you would do:
1924
1925                    LATEST @ HIDDEN
1926
1927            'HIDE word' toggles the flag on a named 'word'.
1928
1929            Setting this flag stops the word from being found by FIND, and so can be used to make 'private'
1930            words.  For example, to break up a large word into smaller parts you might do:
1931
1932                    : SUB1 ... subword ... ;
1933                    : SUB2 ... subword ... ;
1934                    : SUB3 ... subword ... ;
1935                    : MAIN ... defined in terms of SUB1, SUB2, SUB3 ... ;
1936                    HIDE SUB1
1937                    HIDE SUB2
1938                    HIDE SUB3
1939
1940            After this, only MAIN is 'exported' or seen by the rest of the program.
1941    */
1942
1943            defcode "HIDDEN",6,,HIDDEN
1944            pop %edi                 // Dictionary entry.
1945            addl $4,%edi             // Point to name/flags byte.
1946            xorb $F_HIDDEN,(%edi)    // Toggle the HIDDEN bit.
1947            NEXT
1948
1949            defword "HIDE",4,,HIDE
1950            .int WORD                // Get the word (after HIDE).
1951            .int FIND                // Look up in the dictionary.
1952            .int HIDDEN              // Set F_HIDDEN flag.
1953            .int EXIT                // Return.
1954
1955    /*
1956            ' (TICK) is a standard FORTH word which returns the codeword pointer of the next word.
1957
1958            The common usage is:
```

```
        ' FOO ,

        which appends the codeword of FOO to the current word we are defining (this only works in compiled code).

        You tend to use ' in IMMEDIATE words.  For example an alternate (and rather useless) way to define
        a literal 2 might be:

        : LIT2 IMMEDIATE
                ' LIT ,         \ Appends LIT to the currently-being-defined word
                2 ,             \ Appends the number 2 to the currently-being-defined word
        ;

        So you could do:

        : DOUBLE LIT2 * ;

        (If you don't understand how LIT2 works, then you should review the material about compiling words
        and immediate mode).

        This definition of ' uses a cheat which I copied from buzzard92.  As a result it only works in
        compiled code.  It is possible to write a version of ' based on WORD, FIND, >CFA which works in
        immediate mode too.
*/
        defcode "'",1,,TICK
        lodsl                   // Get the address of the next word and skip it.
        pushl %eax              // Push it on the stack.
        NEXT

/*
        BRANCHING ---------------------------------------------------------------------

        It turns out that all you need in order to define looping constructs, IF-statements, etc.
        are two primitives.

        BRANCH is an unconditional branch. 0BRANCH is a conditional branch (it only branches if the
        top of stack is zero).

        The diagram below shows how BRANCH works in some imaginary compiled word.  When BRANCH executes,
        %esi starts by pointing to the offset field (compare to LIT above):

        +--------------------+-------+---- - - ---+------------+------------+---- - - - ----+------------+
        | (Dictionary header) | DOCOL |           | BRANCH     | offset     | (skipped)     | word       |
        +--------------------+-------+---- - - ---+------------+-----|------+---- - - - ----+------------+
                                                                  -  |                      -
                                                                  |  |                      |
                                                                  |  +----------------------+
                                                                 %esi added to offset
```

```
2008          The offset is added to %esi to make the new %esi, and the result is that when NEXT runs, execution
2009          continues at the branch target.  Negative offsets work as expected.
2010
2011          0BRANCH is the same except the branch happens conditionally.
2012
2013          Now standard FORTH words such as IF, THEN, ELSE, WHILE, REPEAT, etc. can be implemented entirely
2014          in FORTH.  They are IMMEDIATE words which append various combinations of BRANCH or 0BRANCH
2015          into the word currently being compiled.
2016
2017          As an example, code written like this:
2018
2019                  condition-code IF true-part THEN rest-code
2020
2021          compiles to:
2022
2023                  condition-code 0BRANCH OFFSET true-part rest-code
2024                                        |             -
2025                                        |             |
2026                                        +-------------+
2027    */
2028
2029          defcode "BRANCH",6,,BRANCH
2030          add (%esi),%esi         // add the offset to the instruction pointer
2031          NEXT
2032
2033          defcode "0BRANCH",7,,ZBRANCH
2034          pop %eax
2035          test %eax,%eax          // top of stack is zero?
2036          jz code_BRANCH          // if so, jump back to the branch function above
2037          lodsl                   // otherwise we need to skip the offset
2038          NEXT
2039
2040    /*
2041          LITERAL STRINGS ---------------------------------------------------------------------
2042
2043          LITSTRING is a primitive used to implement the ." and S" operators (which are written in
2044          FORTH).  See the definition of those operators later.
2045
2046          TELL just prints a string.  It's more efficient to define this in assembly because we
2047          can make it a single Linux syscall.
2048    */
2049
2050          defcode "LITSTRING",9,,LITSTRING
2051          lodsl                   // get the length of the string
2052          push %esi               // push the address of the start of the string
2053          push %eax               // push it on the stack
2054          addl %eax,%esi          // skip past the string
2055          addl $3,%esi            // but round up to next 4 byte boundary
2056          andl $~3,%esi
```

```
        NEXT

        defcode "TELL",4,,TELL
        mov $1,%ebx             // 1st param: stdout
        pop %edx                // 3rd param: length of string
        pop %ecx                // 2nd param: address of string
        mov $ NR_write,%eax     // write syscall
        int $0x80
        NEXT

/*
        QUIT AND INTERPRET ------------------------------------------------------------------------

        QUIT is the first FORTH function called, almost immediately after the FORTH system "boots".
        As explained before, QUIT doesn't "quit" anything.  It does some initialisation (in particular
        it clears the return stack) and it calls INTERPRET in a loop to interpret commands.  The
        reason it is called QUIT is because you can call it from your own FORTH words in order to
        "quit" your program and start again at the user prompt.

        INTERPRET is the FORTH interpreter ("toploop", "toplevel" or "REPL" might be a more accurate
        description -- see: http://en.wikipedia.org/wiki/REPL).
*/

        // QUIT must not return (ie. must not call EXIT).
        defword "QUIT",4,,QUIT
        .int RZ,RSPSTORE // R0 RSP!, clear the return stack
        .int INTERPRET          // interpret the next word
        .int BRANCH,-8          // and loop (indefinitely)

/*
        This interpreter is pretty simple, but remember that in FORTH you can always override
        it later with a more powerful one!
 */
        defcode "INTERPRET",9,,INTERPRET
        call  WORD              // Returns %ecx = length, %edi = pointer to word.

        // Is it in the dictionary?
        xor %eax,%eax
        movl %eax,interpret_is_lit // Not a literal number (not yet anyway ...)
        call  FIND              // Returns %eax = pointer to header or 0 if not found.
        test %eax,%eax          // Found?
        jz 1f

        // In the dictionary.  Is it an IMMEDIATE codeword?
        mov %eax,%edi           // %edi = dictionary entry
        movb 4(%edi),%al // Get name+flags.
        push %ax                // Just save it for now.
        call  TCFA              // Convert dictionary entry (in %edi) to codeword pointer.
        pop %ax
```

```
2106            andb $F_IMMED,%al       // Is IMMED flag set?
2107            mov %edi,%eax
2108            jnz 4f                  // If IMMED, jump straight to executing.
2109
2110            jmp 2f
2111
2112    1:      // Not in the dictionary (not a word) so assume it's a literal number.
2113            incl interpret_is_lit
2114            call  NUMBER            // Returns the parsed number in %eax, %ecx > 0 if error
2115            test %ecx,%ecx
2116            jnz 6f
2117            mov %eax,%ebx
2118            mov $LIT,%eax           // The word is LIT
2119
2120    2:      // Are we compiling or executing?
2121            movl var_STATE,%edx
2122            test %edx,%edx
2123            jz 4f                   // Jump if executing.
2124
2125            // Compiling - just append the word to the current dictionary definition.
2126            call  COMMA
2127            mov interpret_is_lit,%ecx // Was it a literal?
2128            test %ecx,%ecx
2129            jz 3f
2130            mov %ebx,%eax           // Yes, so LIT is followed by a number.
2131            call  COMMA
2132    3:      NEXT
2133
2134    4:      // Executing - run it!
2135            mov interpret_is_lit,%ecx // Literal?
2136            test %ecx,%ecx          // Literal?
2137            jnz 5f
2138
2139            // Not a literal, execute it now.  This never returns, but the codeword will
2140            // eventually call NEXT which will reenter the loop in QUIT.
2141            jmp *(%eax)
2142
2143    5:      // Executing a literal, which means push it on the stack.
2144            push %ebx
2145            NEXT
2146
2147    6:      // Parse error (not a known word or a number in the current BASE).
2148            // Print an error message followed by up to 40 characters of context.
2149            mov $2,%ebx             // 1st param: stderr
2150            mov $errmsg,%ecx        // 2nd param: error message
2151            mov $errmsgend-errmsg,%edx // 3rd param: length of string
2152            mov $__NR_write,%eax    // write syscall
2153            int $0x80
2154
```

```
        mov (currkey),%ecx      // the error occurred just before currkey position
        mov %ecx,%edx
        sub $buffer,%edx        // %edx = currkey - buffer (length in buffer before currkey)
        cmp $40,%edx            // if > 40, then print only 40 characters
        jle 7f
        mov $40,%edx
7:      sub %edx,%ecx           // %ecx = start of area to print, %edx = length
        mov $__NR_write,%eax    // write syscall
        int $0x80

        mov $errmsgnl,%ecx      // newline
        mov $1,%edx
        mov $__NR_write,%eax    // write syscall
        int $0x80

        NEXT

        .section .rodata
errmsg: .ascii "PARSE ERROR: "
errmsgend:
errmsgnl: .ascii "\n"

        .data                   // NB: easier to fit in the .data section
        .align 4
interpret_is_lit:
        .int 0                  // Flag used to record if reading a literal

/*
        ODDS AND ENDS -----------------------------------------------------------------

        CHAR puts the ASCII code of the first character of the following word on the stack.  For example
        CHAR A puts 65 on the stack.

        EXECUTE is used to run execution tokens.  See the discussion of execution tokens in the
        FORTH code for more details.

        SYSCALL0, SYSCALL1, SYSCALL2, SYSCALL3 make a standard Linux system call.  (See <asm/unistd.h>
        for a list of system call numbers).  As their name suggests these forms take between 0 and 3
        syscall parameters, plus the system call number.

        In this FORTH, SYSCALL0 must be the last word in the built-in (assembler) dictionary because we
        initialise the LATEST variable to point to it.  This means that if you want to extend the assembler
        part, you must put new words before SYSCALL0, or else change how LATEST is initialised.
*/

        defcode "CHAR",4,,CHAR
        call  WORD              // Returns %ecx = length, %edi = pointer to word.
        xor %eax,%eax
        movb (%edi),%al         // Get the first character of the word.
```

```
2204            push %eax               // Push it onto the stack.
2205            NEXT
2206
2207            defcode "EXECUTE",7,,EXECUTE
2208            pop %eax                // Get xt into %eax
2209            jmp *(%eax)             // and jump to it.
2210                                    // After xt runs its NEXT will continue executing the current word.
2211
2212            defcode "SYSCALL3",8,,SYSCALL3
2213            pop %eax                // System call number (see <asm/unistd.h>)
2214            pop %ebx                // First parameter.
2215            pop %ecx                // Second parameter
2216            pop %edx                // Third parameter
2217            int $0x80
2218            push %eax               // Result (negative for -errno)
2219            NEXT
2220
2221            defcode "SYSCALL2",8,,SYSCALL2
2222            pop %eax                // System call number (see <asm/unistd.h>)
2223            pop %ebx                // First parameter.
2224            pop %ecx                // Second parameter
2225            int $0x80
2226            push %eax               // Result (negative for -errno)
2227            NEXT
2228
2229            defcode "SYSCALL1",8,,SYSCALL1
2230            pop %eax                // System call number (see <asm/unistd.h>)
2231            pop %ebx                // First parameter.
2232            int $0x80
2233            push %eax               // Result (negative for -errno)
2234            NEXT
2235
2236            defcode "SYSCALL0",8,,SYSCALL0
2237            pop %eax                // System call number (see <asm/unistd.h>)
2238            int $0x80
2239            push %eax               // Result (negative for -errno)
2240            NEXT
2241
2242    /*
2243            DATA SEGMENT ------------------------------------------------------------------
2244
2245            Here we set up the Linux data segment, used for user definitions and variously known as just
2246            the 'data segment', 'user memory' or 'user definitions area'.  It is an area of memory which
2247            grows upwards and stores both newly-defined FORTH words and global variables of various
2248            sorts.
2249
2250            It is completely analogous to the C heap, except there is no generalised 'malloc' and 'free'
2251            (but as with everything in FORTH, writing such functions would just be a Simple Matter
2252            Of Programming).  Instead in normal use the data segment just grows upwards as new FORTH
```

```
2253          words are defined/appended to it.
2254
2255          There are various "features" of the GNU toolchain which make setting up the data segment
2256          more complicated than it really needs to be.  One is the GNU linker which inserts a random
2257          "build ID" segment.  Another is Address Space Randomization which means we can't tell
2258          where the kernel will choose to place the data segment (or the stack for that matter).
2259
2260          Therefore writing this set up data segment assembler routine is a little more complicated
2261          than it really needs to be.  We ask the Linux kernel where it thinks the data segment starts
2262          using the brk(2) system call, then ask it to reserve some initial space (also using brk(2)).
2263
2264          You don't need to worry about this code.
2265   */
2266          .text
2267          .set INITIAL_DATA_SEGMENT_SIZE,65536
2268   set_up_data_segment:
2269          xor %ebx,%ebx           // Call brk(0)
2270          movl $__NR_brk,%eax
2271          int $0x80
2272          movl %eax,var_HERE      // Initialise HERE to point at beginning of data segment.
2273          addl $INITIAL_DATA_SEGMENT_SIZE,%eax     // Reserve nn bytes of memory for initial data segment.
2274          movl %eax,%ebx          // Call brk(HERE+INITIAL_DATA_SEGMENT_SIZE)
2275          movl $__NR_brk,%eax
2276          int $0x80
2277          ret
2278
2279   /*
2280          We allocate static buffers for the return static and input buffer (used when
2281          reading in files and text that the user types in).
2282   */
2283          .set RETURN_STACK_SIZE,8192
2284          .set BUFFER_SIZE,4096
2285
2286          .bss
2287   /* FORTH return stack. */
2288          .align 4096
2289   return_stack:
2290          .space RETURN_STACK_SIZE
2291   return_stack_top:       // Initial top of return stack.
2292
2293   /* This is used as a temporary input buffer when reading from files or the terminal. */
2294          .align 4096
2295   buffer:
2296          .space BUFFER_SIZE
2297
2298   /*
2299          START OF FORTH CODE -----------------------------------------------------------------------
2300
2301          We've now reached the stage where the FORTH system is running and self-hosting.  All further
```

```
        words can be written as FORTH itself, including words like IF, THEN, ." etc which in most
        languages would be considered rather fundamental.

        I used to append this here in the assembly file, but I got sick of fighting against gas's
        crack-smoking (lack of) multiline string syntax.  So now that is in a separate file called
        jonesforth.f

        If you don't already have that file, download it from http://annexia.org/forth in order
        to continue the tutorial.
*/


/* END OF jonesforth.S */
```